

Guía de ejercicios 3 - Media Queries



¡Hola! Te damos la bienvenida a esta nueva guía de ejercicios.

¿En qué consiste esta guía?

En la siguiente guía revisaremos cómo utilizar un breakpoint para cambiar los estilos de un elemento, definiendo también múltiples breakpoints para variar la cantidad de columnas en una grilla diseñada con CSS grid y haciendo desaparecer un elemento en un punto de quiebre.

Finalmente, utilizaremos Flexbox y media queries para cambiar la orientación de elementos, para crear un layout responsivo con CSS grid.

¡Vamos con todo!



Tabla de contenidos

¿Qué es un media query?	3
¿Qué es un breakpoint?	3
Actividad 1: Cambiando la tipografía con media queries	5
Construyendo múltiples breakpoints	5
¿Y cómo funciona para pantallas menores de 576px?	6
Actividad 2: Múltiples cambios	7
Las grillas y los media queries	8
Actividad 3: Columnas y breakpoints	9
Flexbox y los media queries	11
Cambiando la orientación de un menú con media queries	11
Modificando un formulario con media queries	13
Actividad 4: Perfil flexible	15
Diseñando diferentes layouts	16
Actividad 5: Layout abstracto	19
Anatomía avanzada de un breakpoint	19
Tipo de media	19
Operador and	21
Condición	21
Resumen	21



¡Comencemos!

¿Qué es un media query?

Es un bloque de CSS que se ejecuta solo si se cumple una condición, normalmente estas son anchos específicos de pantalla que llamaremos breakpoints.

¿Qué es un breakpoint?

Los breakpoints o puntos de quiebre, en su traducción al español, son cambios estéticos producidos en una página web en el momento que el viewport cumple una condición preestablecida.

Por ejemplo, en nuestro sitio podrías tener una galería de imágenes en una distribución de elementos de 2 columnas, y con un breakpoint declaramos que cuando el ancho sea mayor a 768px, la cantidad de columnas aumente a 3 y se visualicen más imágenes por fila.



Imagen 1. 2 Columnas de imágenes.
Fuente: Desafío Latam.



Imagen 2. 3 Columnas de imágenes al aplicarse un breakpoint.

Fuente: Desafío Latam.

Ahora te estarás preguntando, ¿y cómo se construye un breakpoint? Para entenderlo, ocupa el siguiente código en tu editor preferido y cambia el ancho de pantalla para visualizar cómo cambia el color de fondo.

```
@media (max-width: 576px) {  
  body {  
    background: black;  
  }  
}  
  
@media (min-width: 576px) {  
  body {  
    background: #749c2c;  
  }  
}
```

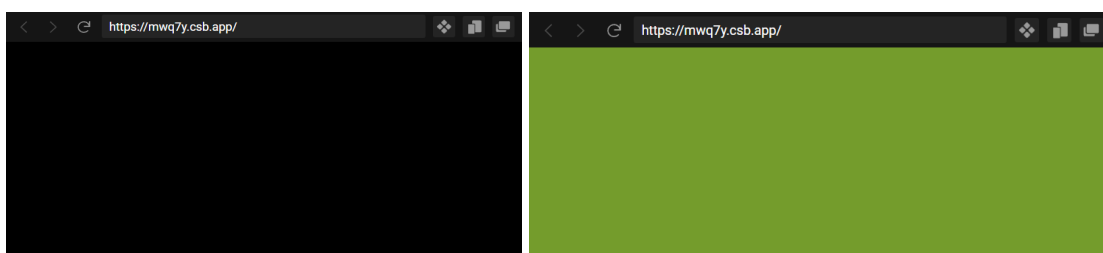


Imagen 3. Cambio de color de fondo en diferentes anchuras de pantalla.

Fuente: Desafío Latam.

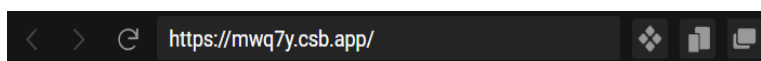
La declaración de un breakpoint es a través de un *@media* acompañado de parámetros que especifican que tipo de *media* se va a ocupar y la condición que se debe cumplir.



Actividad 1: Cambiando la tipografía con media queries

Realicemos juntos un ejercicio en donde cambiamos la tipografía de un párrafo utilizando un breakpoint. Sigue los siguientes pasos:

1. Crea un archivo nuevo con una base de HTML.
2. Crea una etiqueta de párrafo con un texto genérico.
3. Utilizando CSS, crea un breakpoint para el tamaño extra small (<576px) que defina un font-family *monospace*.
4. Crea un breakpoint para el tamaño small (>=576px) que defina un font-family: *fantasy*.
5. Observa el cambio en el inspector de elementos.



Lorem ipsum dolor sit amet consectetur adipisicing elit. Minus, possimus? Ipsa, sint, vel quidem non neque nisi aliquid, est voluptate dicta ducimus quae beatae officia? Laborum magnam dolorum animi vel?



Lorem ipsum dolor sit amet consectetur adipisicing elit. Minus, possimus? Ipsa, sint, vel quidem non neque nisi aliquid, est voluptate dicta ducimus quae beatae officia? Laborum magnam dolorum animi vel?

Imagen 4. Cambio de tipografía a partir de un breakpoint.
Fuente: Desafío Latam.

Construyendo múltiples breakpoints

Ahora que sabemos que son y cómo ocupar breakpoints, nos podemos preguntar ¿cuántos breakpoints debemos definir? La respuesta siempre dependerá del objetivo de tu sitio web, lo importante es entender que no es necesario declarar todos los breakpoints si estos no aportan un valor importante a nuestro diseño.

Podemos elegir las medidas que queramos para nuestros breakpoints y estos serán independientes a los estilos que declaremos dentro, no obstante, podemos basarnos en frameworks como Bootstrap donde se definen los siguientes breakpoints.

Extra small	Small	Medium	Large	Extra large
<576px	≥576px	≥768px	≥992px	≥1200px

Imagen 5. Diferentes medidas para la definición de breakpoint.

Fuente: [Bootstrap](#).

A continuación, revisemos el código de un ejercicio en donde con diferentes breakpoints le cambiamos el color de texto a una etiqueta `h3`.

```
h3 { color: crimson }

@media (min-width: 576px) {
  h3 { color: purple }
}

@media (min-width: 768px) {
  h3 { color: green }
}

@media (min-width: 992px) {
  h3 { color: chocolate }
}
```

Desafío Latam → **Desafío Latam** → **Desafío Latam** → **Desafío Latam**

Imagen 6. Cambio de colores a partir varios breakpoints.

Fuente: Desafío Latam.

Como puedes notar, podemos declarar todos los breakpoints que queramos y dentro definir todos los cambios que necesitemos.

¿Y cómo funciona para pantallas menores de 576px?

El breakpoint que declaramos con un ancho mínimo de 576px realmente no es necesario y puede omitirse, esto debido a que los estilos que escribimos en nuestro documento CSS serán asignados a los elementos, independientemente de los media queries que declaramos. Es decir, que podemos diseñar nuestro sitio web pensando inicialmente en los dispositivos móviles y luego con los breakpoints, empezamos a considerar las diferentes medidas de pantalla.

A continuación, te mostramos qué sucede con la asignación de una propiedad a un elemento cuando la condición de un breakpoint se cumple y este también está involucrado con la misma propiedad .

```
@media (min-width: 576px)
h3 {
  color: purple;
}

h3 {
  color: crimson;
}
```

Imagen 7. Cambio de colores a partir varios breakpoints.
Fuente: Desafío Latam.

Como puedes apreciar, el navegador prioriza las declaraciones del media query y tacha o ignora la asignación original de esta propiedad. Las propiedades que no se estén asignando, mantendrán su efecto en el elemento.



Actividad 2: Múltiples cambios

Además de modificar el valor de una misma propiedad, podríamos cambiar o asignar otras propiedades del mismo elemento. En este ejercicio, nuestro objetivo será asignar color de fondo, de texto y bordes redondeados a un botón en diferentes breakpoints; sin embargo, asignaremos un *display: none* en un último quiebre para eliminarlo de la vista.

Los pasos a seguir son los siguientes:

1. Crea un botón en el HTML con un texto dentro que diga *enviar*.
2. Define un color de fondo al botón.
3. Crea un breakpoint para la medida *small* que asigne bordes redondeados de 15px.
4. Crea un breakpoint para la medida *medium* que asigne un color de texto al botón.
5. Crea un tercer y último breakpoint para eliminar el botón de la vista con un *display: none*.



Imagen 8. Cambios de estilos progresivos con breakpoints.
Fuente: Desafío Latam.

Las grillas y los media queries

Los media queries son usados principalmente para los cambios de layout en una página, con la ayuda de herramientas como CSS Grid y Flexbox, podemos generar cambios importantes que ayuden y mejoren la experiencia de nuestros usuarios.

Vamos a realizar un ejemplo muy típico, diseñar una grilla que cambie la cantidad de columnas en función del ancho de la pantalla. Para esto, abre tu editor de código preferido y copia el siguiente fragmento HTML.

```
<div class="grid">
  <p>Desafío latam</p>
  <p>Desafío latam</p>
  <p>Desafío latam</p>
  <p>Desafío latam</p>
</div>
```

Ahora definamos los estilos base (versión mobile) de nuestros elementos HTML, incluyendo la definición de una grilla con CSS GRID que declare 1 columna con la *propiedad* `grid-template-columns`.

```
p {
  background: #749c2c;
  color: white;
  padding: 10px;
  margin: 10px;
}
.grid {
  display: grid;
  grid-template-columns: 1fr;
}
```

Si visualizamos el navegador deberías poder ver lo siguiente:



Imagen 9. Una columna de elementos con CSS GRID (versión Móvil).
Fuente: Desafío Latam.

Ahora que tenemos lista la versión de 1 columna, empezamos a declarar los breakpoints de los demás anchos de pantalla, en este caso solo un par de media queries.

```
@media (min-width: 576px) {  
  .grid { grid-template-columns: 1fr 1fr }  
}  
  
@media (min-width: 768px) {  
  .grid { grid-template-columns: 1fr 1fr 1fr 1fr }  
}
```

El resultado en las diferentes variantes de ancho sería entonces el siguiente:

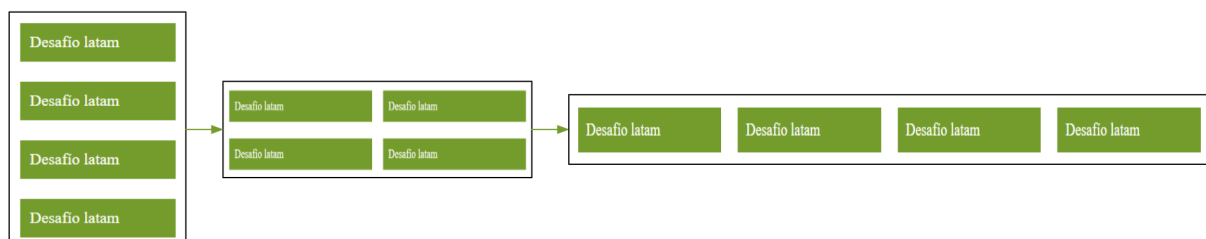


Imagen 10. Cambios de columnas progresivos con breakpoints.
Fuente: Desafío Latam.

¡Ahí lo tenemos! Ahora podemos variar la cantidad de columnas a partir de breakpoints y así generar diferentes layouts en nuestra página.



Actividad 3: Columnas y breakpoints

Entre los usos más populares de CSS Grid se encuentra el diseño o maquetación de una galería de imágenes, y ahora que hemos aprendido a ocupar los breakpoints pongamos a prueba nuestra nueva habilidad con un ejercicio en donde deberemos definir 3 medidas diferentes: Extra small, small, medium y large, para presentar una galería de imágenes de perritos obtendremos de [Place Dog](https://place.dog/).

La URL que debes formar para obtener una imagen de un perrito es la siguiente:

<https://placedog.net/200/200>

Los números representan las dimensiones de ancho de alto de la imagen. Una vez que ya hayas elegido qué imágenes vas a ocupar, sigue los siguientes pasos:

1. Crea un DIV en el HTML que sirva como contenedor de una grilla de CSS.
2. Incluye 6 imágenes que muestren imágenes de perritos de Place God.
3. Con CSS Grid diseña la versión móvil de 1 sola columna.
4. Escribe 3 breakpoints para las versiones small, medium y large que varíe la cantidad de columnas.

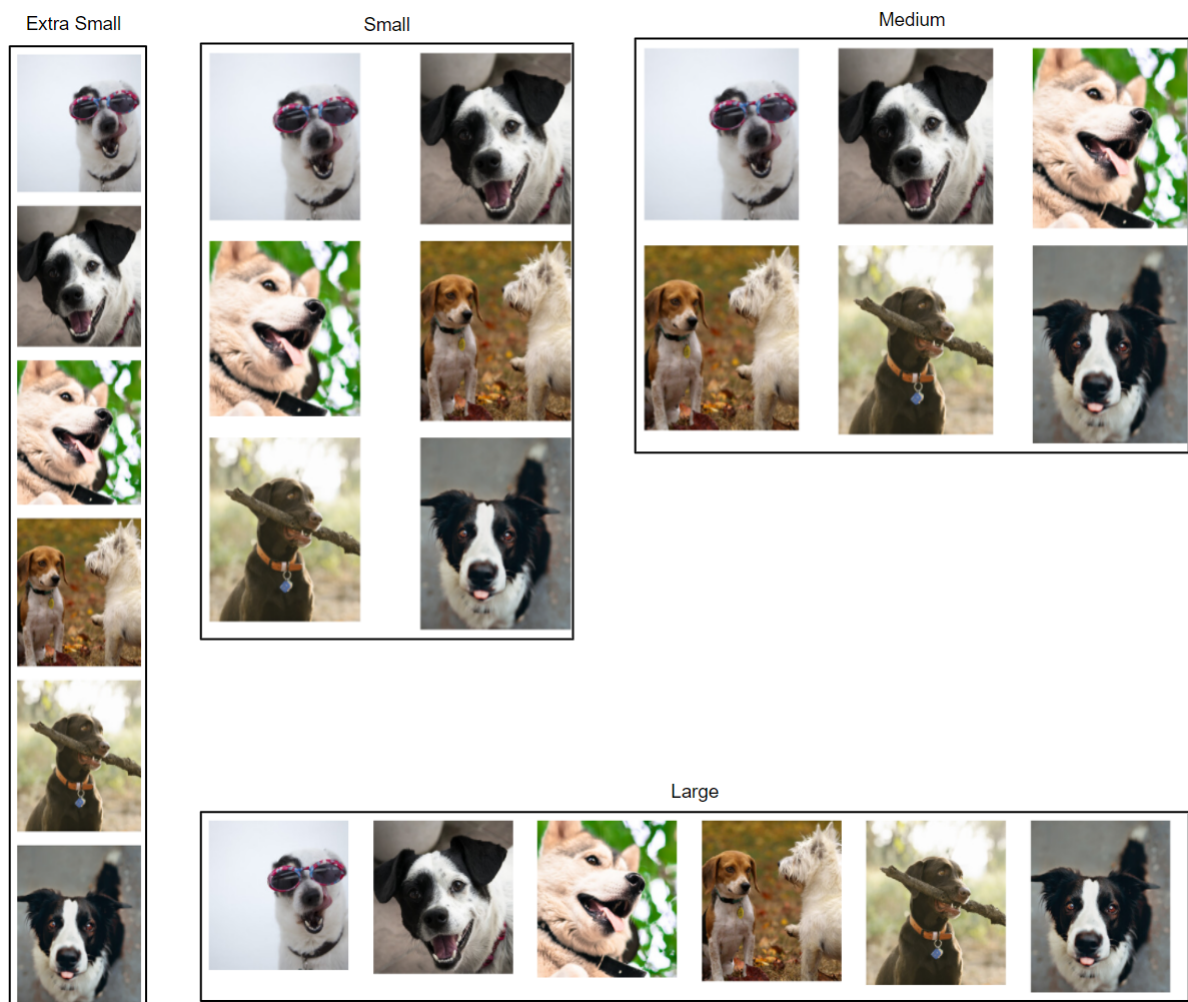


Imagen 11. Diferentes vistas para galería de imágenes de perritos.

Fuente: Desafío Latam.

Flexbox y los media queries

Así como podemos modificar la cantidad de columnas en una grilla diseñada con CSS Grid, también es posible modificar las propiedades de un contenedor flexible, por ejemplo, la orientación o la distribución horizontal.

Cambiando la orientación de un menú con media queries

Realicemos un ejercicio para ver un ejemplo de esto, en donde diseñaremos un menú de navegación con opciones verticales para versión de teléfono y horizontales para escritorios.

En el editor de códigos, agrega la base de HTML y dentro del body el siguiente fragmento:

```
<nav>
  <ul>
    <li>Inicio</li>
    <li>Sobre nosotros</li>
    <li>Contacto</li>
  </ul>
</nav>
```

Ahora, definamos los estilos base para la versión de teléfono que ubique las opciones de forma vertical utilizando la orientación *column* de flexbox:

```
ul {
  padding: 10px;
  list-style: none;
  color: white;
  background: #749c2c;
  display: flex;
  flex-direction: column;
}

ul li {
  padding: 5px;
}
```

Para este punto deberás poder visualizar en el navegador lo siguiente:



Inicio
Sobre nosotros
Contacto

Imagen 12. Menú con Flexbox versión de teléfonos.
Fuente: Desafío Latam.

Muy bien, acá tenemos un simple menú con 3 opciones ubicadas de forma vertical, lo siguiente será escribir en un breakpoint que en tamaño *small* ubique las opciones horizontalmente y distribuya el espacio disponible entre los elementos, esto lo haremos cambiando la orientación del contenedor flexible y aplicando la propiedad *justify-content*.

```
@media (min-width: 576px) {  
  ul {  
    flex-direction: row;  
    justify-content: space-around;  
  }  
}
```

Ajusta el ancho de tu navegador para entrar al punto de quiebre y deberás ver el menú de la siguiente manera:



Inicio Sobre nosotros Contacto

Imagen 13. Menú con Flexbox versión small.
Fuente: Desafío Latam.

¡Perfecto! Hemos logrado cambiar la orientación de las opciones a partir de un punto de quiebre, ahora escribamos un media query que a partir de la vista *medium* ubique las opciones a la derecha cambiando el valor de la propiedad *justify-content*.

```
@media (min-width: 768px) {  
  ul {  
    justify-content: flex-end;  
  }  
}
```



Inicio Sobre nosotros Contacto

Imagen 14. Menú con Flexbox versión medium.
Fuente: Desafío Latam.

Ahí lo tenemos, un menú de navegación que varía la distribución de elementos utilizando flexbox y puntos de quiebre.

Modificando un formulario con media queries

Ahora realicemos otro ejercicio donde tengamos un pequeño formulario de filtros de búsquedas que se presente forma vertical en versión de teléfono y cambie la orientación en versión de escritorio.

Para esto ocupa el siguiente código HTML con los elementos relacionados con los filtros de búsqueda.

```
<section class="filtros">
  <div>
    <p>Buscar por nombre:</p>
    <input type="text" />
  </div>
  <div>
    <p>Ordenar por:</p>
    <select>
      <option value="precio">Precio</option>
      <option value="fecha">Fecha</option>
    </select>
  </div>
  <div>
    <p>Mostrar solo artículos:</p>
    Nuevos <input type="checkbox" />
    Usados <input type="checkbox" />
  </div>
</section>
```

Y el siguiente código CSS, que define los estilos en la versión base o móvil:

```
.filtros {
  text-align: center;
  display: flex;
  flex-direction: column;
}

.filtros div {
  margin-bottom: 30px;
}
```

```
}
```

Ahora revisa el navegador y deberás visualizar lo siguiente:

Buscar por nombre

Ordenar por

Precio ▾

Mostrar solo artículos

Nuevos: ☐ Usados ☐

Imagen 15. Filtro de búsqueda versión Mobile.
Fuente: Desafío Latam.

Ahora procedemos a escribir el punto de quiebre para que la orientación de los elementos flexibles cambie y se ubiquen los elementos de forma horizontal, además de distribuir el espacio disponible entre ellos.

```
@media (min-width: 576px) {  
  .filtros {  
    flex-direction: row;  
    justify-content: space-around;  
  }  
}
```

Finalmente, recargamos el navegador asegurándonos que estamos en el ancho desktop y tendremos el siguiente resultado.

Buscar por nombre:

Ordenar por:

Precio ▾

Mostrar solo artículos:

Nuevos ☐ Usados ☐

Imagen 16. Filtro de búsqueda versión Desktop.
Fuente: Desafío Latam.



Actividad 4: Perfil flexible

En esta actividad el objetivo será usar flexbox para maquetar la siguiente imagen que representa la versión de teléfono:

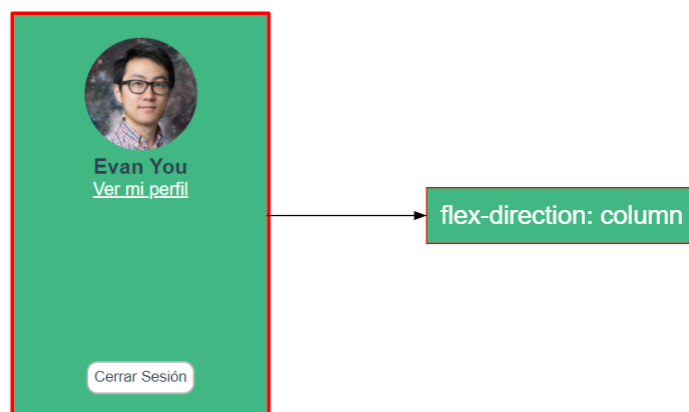


Imagen 17. Perfil flexible Mobile.

Fuente: Desafío Latam.

Y posteriormente, escribir un breakpoint que contenga un cambio de estilos para lograr cambiar la presentación en la versión desktop para que quede de la siguiente manera:

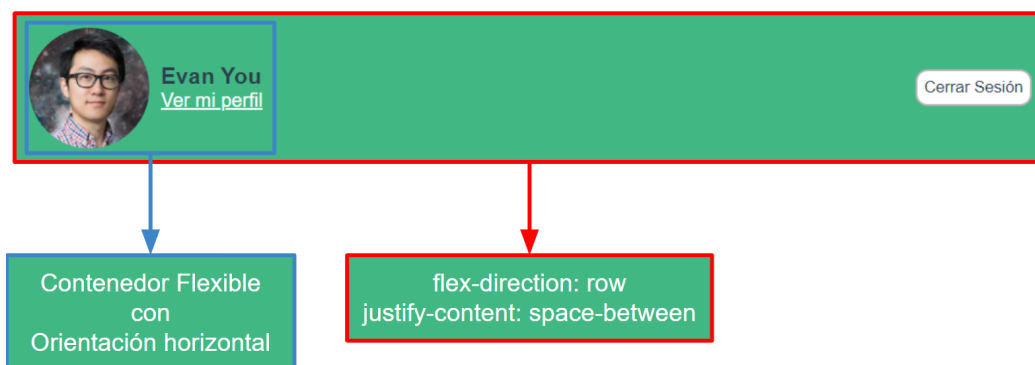


Imagen 18. Perfil flexible Desktop.

Fuente: Desafío Latam.

Para esto deberás seguir los siguientes pasos:

1. Crea los elementos necesarios en el HTML.
2. Diseña los estilos base usando flexbox para la orientación vertical de los elementos.
3. Escribe un punto de quiebre que cambie la orientación de los elementos.
4. Realiza los ajustes estéticos necesarios para igualar tu resultado a la imagen de referencia.

Diseñando diferentes layouts

Una práctica muy útil y fácil de ocupar para diseñar diferentes layouts en nuestras páginas web, es mezclar los breakpoints con la propiedad *grid-template-areas* de CSS Grid. Veamos un ejemplo en donde tenemos declarado en un HTML diferentes secciones y las vamos ubicando con un template definido de CSS GRID.

Utiliza el siguiente fragmento de código HTML:

```
<div>
  <nav>Menú de navegación</nav>
  <main>Sección principal</main>
  <footer>Pie de página</footer>
</div>
```

Ahora definamos los estilos base de cada sección y la asignación de su nombre para *template area* que declararemos más adelante.

```
nav {
  padding: 10px;
  grid-area: menu;
  background: #0037a1;
  color: white;
}
main {
  grid-area: seccion-principal;
  padding: 10px;
}
footer {
  grid-area: pie-de-pagina;
  background: #d02a1e;
  color: white;
  padding: 10px;
}
```


Con este código, tu página web debería verse de la siguiente manera:



Imagen 19. Versión base.
Fuente: Desafío Latam.

Ahora procedamos a definir la grilla con CSS GRID de nuestras secciones utilizando *grid-template-rows* y *grid-template-areas*:

```
div {  
  display: grid;  
  border: 1px solid black;  
  grid-template-rows: 1fr 10fr 1fr;  
  grid-template-areas:  
    "menu"  
    "seccion-principal"  
    "pie-de-pagina";  
}
```

De esta manera ya tendríamos el template para la versión mobile, quedando de la siguiente manera:



Imagen 20. Versión base (mobile).
Fuente: Desafío Latam.

Ahora debemos ocuparnos en escribir el breakpoint que cambie el layout por una distribución más acorde a la versión Desktop, esta tendrá 2 columnas en vez de 1 y cambiará la ubicación de los elementos para presentar una interfaz acorde a las tendencias de los últimos años, donde encontramos menús de navegación verticales.

Utilizando el siguiente código:

```
@media (min-width: 992px) {  
  div{  
    grid-template-areas:  
      "menu seccion-principal"  
      "menu seccion-principal"  
      "menu pie-de-pagina";  
    grid-template-columns: 6rem 1fr;  
  }  
}
```

Obtendremos el siguiente resultado:



Imagen 21. Versión Desktop.
Fuente: Desafío Latam.

Maravilloso, utilizando CSS Grid hemos logrado en pocas líneas diseñar 2 layouts que varían según el ancho de la pantalla a través de media queries. Además, ocupamos la unidad *fr* para definir el tamaño de una columna en función al espacio restante disponible.



Actividad 5: Layout abstracto

Utiliza CSS Grid y media queries para diseñar las 3 vistas de la siguiente galería de imágenes:



Imagen 22. Galería de imágenes.

Fuente: Desafío Latam.

Para esto deberás seguir los siguientes pasos:

1. Crea los elementos necesarios en el HTML.
2. Utiliza CSS Grid para diseñar el layout en versión Mobile.
3. Escribe un media query para diseñar el layout de versión Tablet.
4. Escribe un media query para diseñar el layout de versión Desktop.

Anatomía avanzada de un breakpoint

Los media queries se escriben en base a una sintaxis definida por el lenguaje, esta contiene el tipo de media, el parámetro condicional y la medida de la condición.

```
@media [tipo de media] [operador and] [condición]
```

Tipo de media

Los diferentes tipos de media y sus usos son los siguientes:

- **all:** Aplica el cambio a todos los medias.

- **screen:** Es la más utilizada y se enfoca en la pantalla del dispositivo.
- **speech:** Se utiliza para los sintetizadores de voz.
- **print:** Diseño dedicado a la impresión de un documento. Es posible diseñar cambios estéticos en un sitio web dedicado a la impresión o exportación en PDF.

Un ejemplo de este último tipo de media lo puedes visualizar rápidamente ocupando el siguiente código:

```
<h1>Desafío Latam</h1>

<style>
  @media print and (min-width: 480px) {
    h1 {
      color: #7ba238;
      font-family: fantasy;
      font-size: 150px;
    }
  }
</style>
```

En donde se crea una etiqueta h1 y se declara con un *@media print* que el título tendrá otra tipografía, otro tamaño y otro color.

Ahora intenta imprimir la página web y verás lo siguiente:

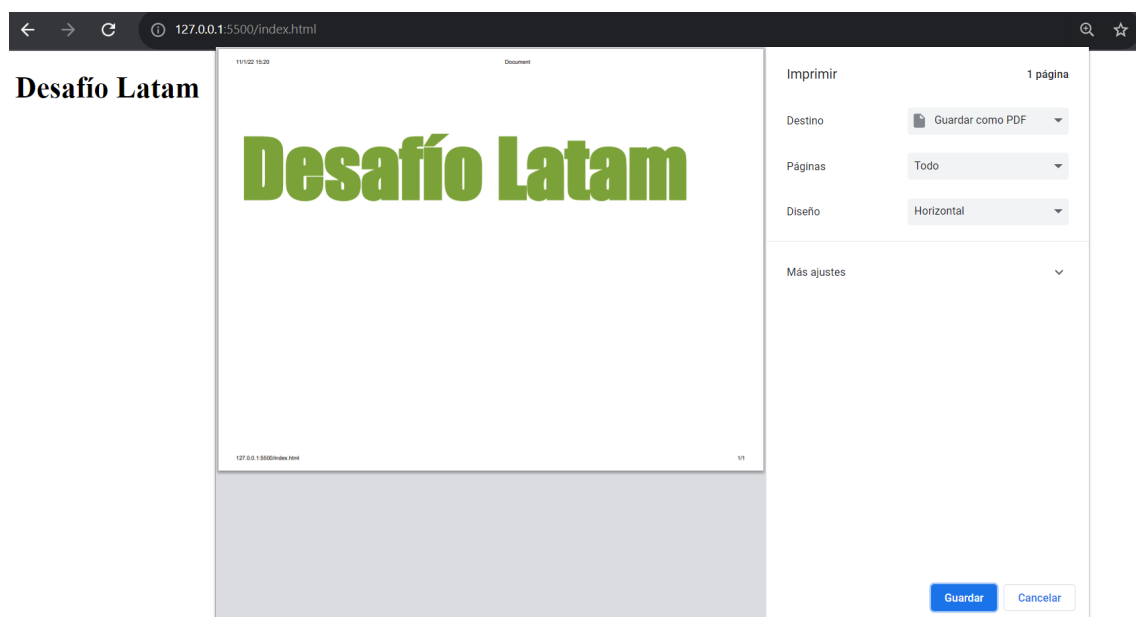


Imagen 23. Media Print.
Fuente: Desafío Latam.



Si quieres conocer más sobre los tipos de media, consulta la [documentación oficial](#) de Developer Mozilla.

Operador and

Es posible escribir un breakpoint pensando en un rango píxeles en vez de un punto de partida o tope. Por ejemplo, en el siguiente breakpoint se especifica que el bloque de estilos se aplicará entre los 320px y 480px:

```
@media screen and  
(min-device-width: 320px) and (max-device-width: 480px) {  
  
  /* ...Estilos... */  
  
}
```

El operador *and* nos ayudará a unir los diferentes criterios o condiciones.

Condición

La condición puede variar entre varias propiedades disponibles en los media queries, aunque el más utilizado es el relacionado con el ancho de pantalla fijando un punto de partida con *min-width* o un punto tope con *max-width*. No obstante, también es posible identificar la orientación del dispositivo (landscape o portrait) y ejecutar cambios a partir de esto.



Si quieres saber más sobre las propiedades que se pueden ocupar en la condición, visita la [documentación](#) de W3School sobre media queries.



Resumen

- En CSS podemos declarar puntos de quiebres con la regla o sentencia *@media* y generar diferentes cambios en nuestra página web a partir del ancho de la pantalla.
- Existen diferentes medidas relacionadas con los diferentes dispositivos que ofrecen una navegación web, y frameworks como Bootstrap declaran o recomiendan unidades específicas para marcar los puntos de quiebre entre dispositivos.

- Los breakpoints nos permiten definir diferentes cambios de estilos en diferentes elementos e inclusive podemos desaparecerlos para mejorar la experiencia de usuario.
- Podemos mezclar los breakpoints con herramientas como flexbox y CSS grid para redistribuir los elementos en función del ancho de la pantalla e incluso modificar el layout.
- Es recomendable escribir los media queries de menor a mayor, pensando siempre en Mobile First y progresivamente los demás dispositivos.